

11.2. 射影追跡回帰(projection pursuit regression, PPR)

射影追跡回帰は回帰モデルの一種で、以下の式で表される。

ただし、入力 X は縦ベクトル、 ω_m は縦ベクトル、 M は**非線形関数の数**である。

また、関数 g_m は**事前に形状は定められておらず**、平滑化手法によって推定される。

$$f(X) = \sum_{m=1}^M g_m(\omega_m^T X) \quad (11.1)$$

特徴

- 説明変数に**平滑化関数を適用する前に**、最適な方向における**行列を最初に予測すること**で加法モデルを拡張
- リッジ関数** $g_m(\omega_m^T X)$ は**非線形関数**で、うまく取れば M が大きくても任意の精度で近似可能

例外として、非線形の関数の個数 $M = 1$ の場合は、**単一指標モデル**と呼ばれる。
このモデルは、線型回帰を**わずかに一般化した**ものと考えられ、以下の式で表せる。

$$f(X) = g_1(\omega_1^T X)$$

訓練データ (x_i, y_i) ($i = 1, 2, \dots, N$) に対して当てはめを行うには、式(11.2)の誤差関数を利用する。

リッジ関数 g_m 及び重み ω_m を **近似的に最小化** すれば良い。

ただし、過学習を避けるために g_m の **複雑度に関する制約は必須** である。

$$\sum_{i=1}^N \left[y_i - \sum_{m=1}^M g_m(\omega_m^T x_i) \right]^2 \quad (11.2)$$

射影追跡回帰の手順は以下のとおりである。

1. 標本の準備

入力データを**中心化/球状化** (まとめて**白色化**と言う)

2. 重み ω が収束するまで更新する

射影追跡回帰のメリット

1. **予測**に有用
2. **非線形関数**で表現可
→ 線形回帰では見逃す関係を捕捉可
3. 次元の呪いの回避
→ 高次元データで利用可

射影追跡回帰のデメリット

- 解釈性に乏しい

11.3. ニューラルネットワーク(NN)

NNとは、人間の脳神経系のニューロンを数理モデル化したもの。

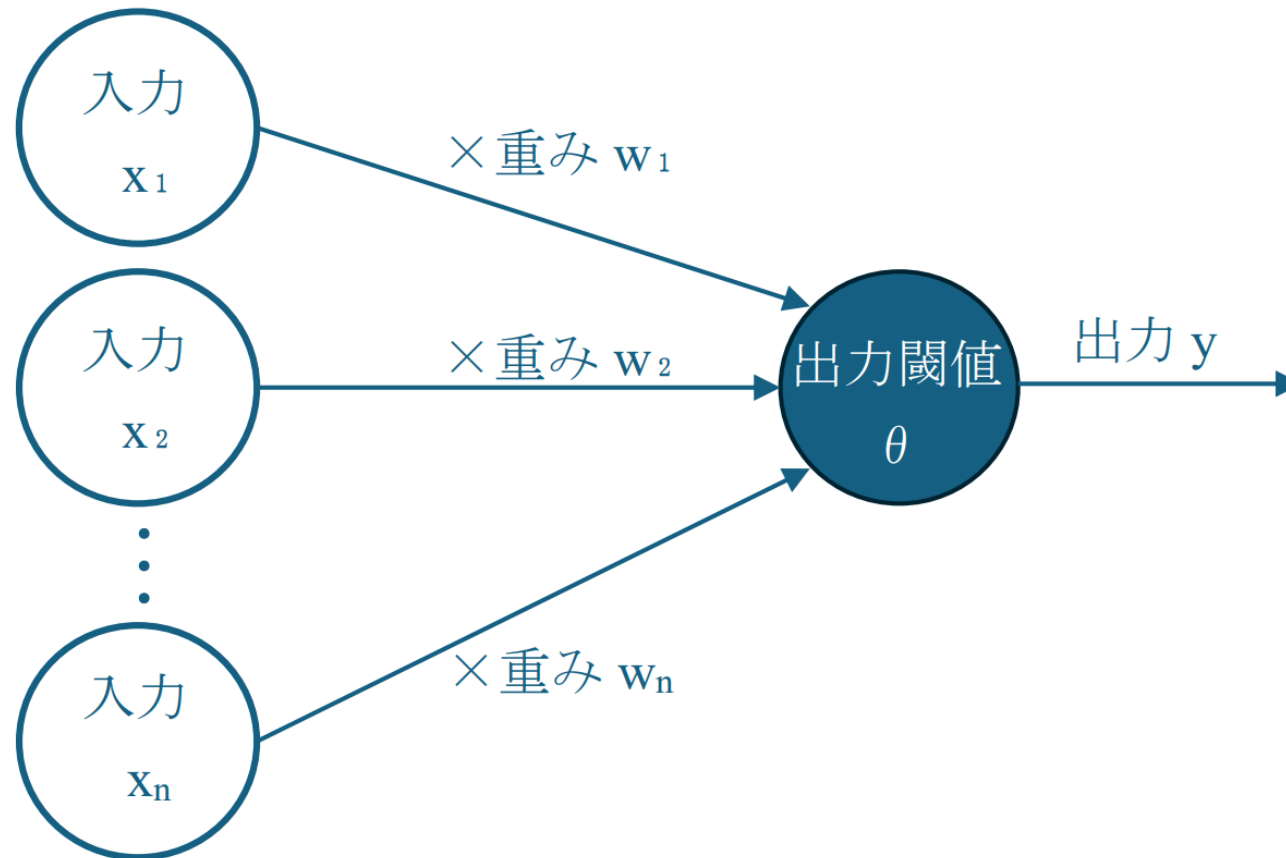
これは、**複数の段階で構成される回帰/分類モデル**と考えることができる。

特徴

- 近いモデルである**パーセプトロン**では解けなかった**線形分離不可**な問題に対応可
- NNは**活性化関数 $\sigma(v)$ の総和**を考えるため射影追跡回帰の特別な場合と考えられる

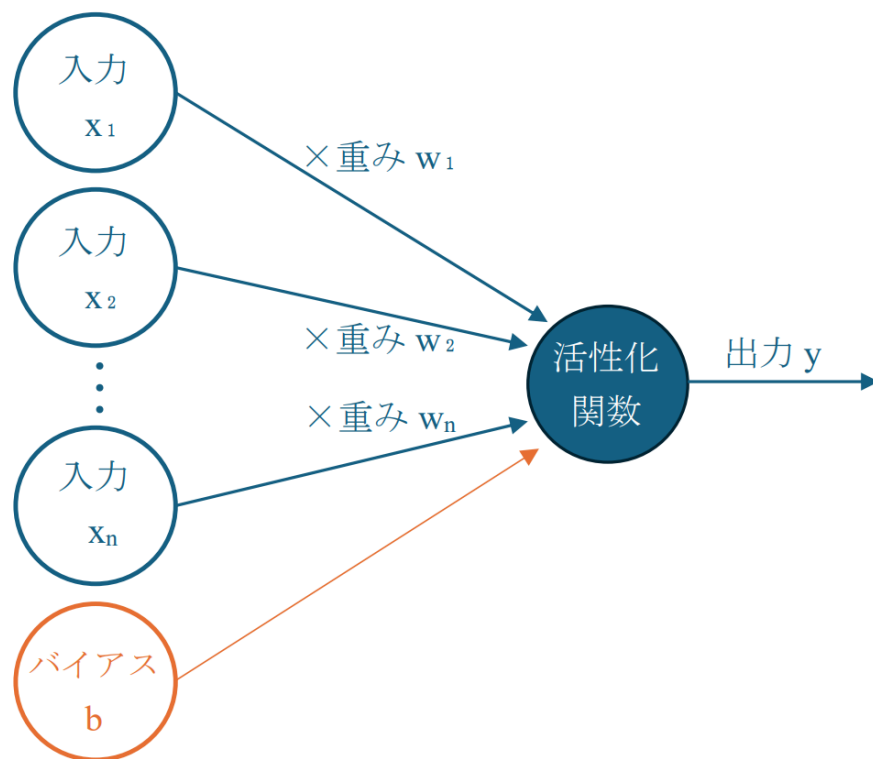
下図の球体を**ユニット**及び**ニューロン**、ユニット間を結ぶ線を**シナプス**という。

ニューロンモデルとは、各入力値 x_n に対応する**重み** w_n をかけた総和が閾値 θ を超えるかどうかを調べるものである。

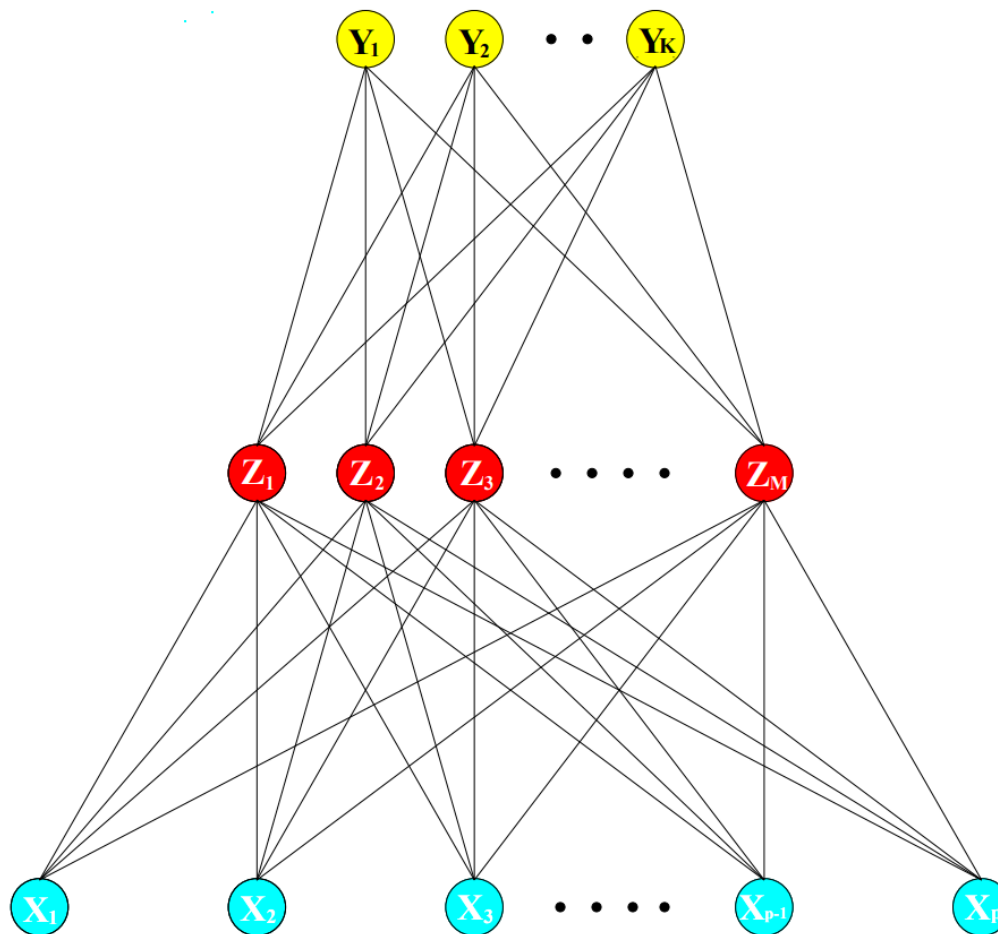


パーセプトロンはニューロンモデルに似ており、各入力 x_n と対応する重み w_n の積に、**バイアス** b_n を加えたものの総和が**0以上か未満か**を調べるものである。
出力は**活性化関数** σ を使って書くことができる。

$$y = \sigma \left(\sum_{i=1}^n x_i w_i + b_i \right)$$



下図がNNである。入力を受け取る X_p のある1層目を**入力層**、結果を出力する Y_K のある最後の層を**出力層**、その間の導出特徴量 Z_M のある全ての層を**中間層**という。



NNは、多数のパーセプトロンを集めて繋げたものだと考えられる。
この際、活性化関数としては以下の3種類が使われることが多い。

1. **シグモイド関数** $\sigma(v) = 1/(1 + \exp(-v))$
2. **ReLU関数** $f(x) = \begin{cases} 0 & (0 > x) \\ x & (0 \leq x) \end{cases}$
3. **動径基底関数** ... 距離のみに基づいて値が決まる **動径関数** の線形和

前ページの導出特徴量 Z_M 、目的変数 Y_K は以下の式で表される。

$$\begin{aligned} Z_m &= \sigma(\alpha_{0m} + \alpha_m^T X), \quad (m = 1, \dots, M) \\ T_k &= \beta_{0k} + \beta_k^T Z, \quad (k = 1, \dots, K) \\ Y_k &= f_k(X) = g_k(T), \quad (k = 1, \dots, K) \end{aligned} \tag{11.5}$$

ただし、 $Z = (Z_1, Z_2, \dots, Z_M)^T$ 、 $T = (T_1, T_2, \dots, T_K)^T$ である。

また、**出力関数** g_k は**出力ベクトル** T を**最終形態** Y に変換する役割がある。
これには、**ソフトマックス関数**が使われることが多い。

$$g_k(T) = \frac{e^{T_k}}{\sum_{l=1}^K e^{T_l}} \tag{11.6}$$

11.4. NNの当てはめ

NNの**重み(weight)**は**未知**であり、訓練データへの当てはめのために**重みを求める必要**がある。

全ての重みを θ とし、構成要素は以下のようにする。

$$\begin{aligned} &\{\alpha_{0m}, \alpha_m; m = 1, 2, \dots, M\} \\ &\{\beta_{0k}, \beta_k; k = 1, 2, \dots, K\} \end{aligned} \tag{11.8}$$

当てはまりの度合いの尺度として、**誤差関数** $R(\theta)$ を導入する。

- 誤差関数とは、予測値と真の値の誤差

回帰の場合には、として**誤差の2乗和**が使われる。

$$R(\theta) = \sum_{k=1}^K \sum_{i=1}^N (y_{ik} - f_k(x_i))^2$$

分類の場合には、前述の誤差の2乗和か**交差エントロピー**が使われる。

$$R(\theta) = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log f_k(x_i)$$

分類機が $G(x) = \operatorname{argmax}_k f_k(x)$ で設計される。

活性化関数と誤差関数に**ソフトマックス関数と交差エントロピー**を使うと、隠れ層を入力とした**ロジスティック回帰**と等価である。

→ 全ての未知パラメータを**最尤法で推定**可能に

ただし、誤差関数 $R(\theta)$ の大域最適解は必要なく、任意の正則化を行えば良い。

→ 訓練データに**過学習**しているため

予測値と真の値の誤差である誤差関数は最小化が必要であり、一般的に**勾配降下法**が使われる。

特にNNでは、**逆伝播**または**誤差逆伝播法**等と呼ばれる。

誤差関数は微分可能なので、容易に勾配が計算できる。

よってネットワーク上を**前向き/後ろ向き**に走査すればよい。

誤差逆伝播法は、更新式を使い**誤差関数を調整**する工程を**複数回**行う。
まず、誤差関数 $R(\theta)$ を書き直す。

$$\begin{aligned} R(\theta) &\equiv \sum_{i=1}^N R_i \\ &= \sum_{i=1}^N \sum_{k=1}^K (y_{ik} - f_k(x_i))^2 \end{aligned} \quad (11.11)$$

荷重についての導関数は、以下の様に得られる。

$$\begin{aligned} \frac{\partial R_i}{\partial \beta_{km}} &= -2(y_{ik} - f_k(x_i))g'_k(\beta_m^T z_i)z_{mi} \\ \frac{\partial R_i}{\partial \alpha_m} &= -2 \sum_{k=1}^K (y_{ik} - f_k(x_i))g'_k(\beta_m^T z_i)\beta_{km}\sigma(\alpha_m^T x_i)x_{il} \end{aligned} \quad (11.12)$$

勾配降下法に基づいて、 $(r + 1)$ 番目の更新式は以下のように表される。
ただし、 γ_r は**学習率**である。

$$\begin{aligned}\beta_{km}^{(r+1)} &= \beta_{km}^{(r)} - \gamma_r \sum_{i=1}^N \frac{\partial R_i}{\partial \beta_{km}^{(r)}} \\ \alpha_{ml}^{(r+1)} &= \alpha_{ml}^{(r)} - \gamma_r \sum_{i=1}^N \frac{\partial R_i}{\partial \alpha_{ml}^{(r)}}\end{aligned}\tag{11.13}$$

また、誤差関数の各偏微分を**現時点でのユニットでの誤差** δ_{ki} 、 s_{mi} を使って表す。

$z_{mi} = \sigma(\alpha_{0m} + \alpha_m^T x_i)$ と定義すると、

$$\begin{aligned}\frac{\partial R_i}{\partial \beta_{km}} &= \delta_{ki} z_{mi} \\ \frac{\partial R_i}{\partial \alpha_{ml}} &= s_{mi} x_{il}\end{aligned}\tag{11.14}$$

δ_{ki} , s_{mi} の2種の誤差から**逆伝播等式**を求められる。

$$s_{mi} = \sigma'(\alpha_m^T x_i) \sum_{k=1}^K \beta_{km} \delta_{ki}\tag{11.15}$$

(11.13)の更新式と逆伝播等式を使うことで、**2段階の走査のアルゴリズム**として実装できる。

1. 前向き走査

現時点の重みを**固定**して、逆伝播等式より関数の**予測値** $\hat{f}(x)$ を計算

2. 後ろ向き走査

出力ユニットの誤差項 δ_{ki} を計算し、**逆伝播等式に代入**し誤差を逆伝播

同様に、隠れユニットの誤差項 s_{mi} を計算し、(11.14)式に代入して**勾配を計算**

この1と2を繰り返すことで、誤差関数を最小化できる。

(11.13)の更新式は、**バッチ学習**で**全訓練データ**を元にパラメータが更新される。
この学習方法では、学習率 γ_r は定数とすることが多い。

一方で、各時点で訓練データ毎に**その場で勾配を更新する**作業を全訓練データにおいて何度か繰り返す**オンライン学習**も可能である。

この際、全訓練データに対して1回の学習の走査を**訓練エポック**と呼ぶ。
この学習方法では、学習率 γ_r を減らしていくことが妥当である。

- 具体的には $r \rightarrow \infty$ に近似した際、 $\gamma_r \rightarrow 0, \sum_r \gamma_r = \infty \cap \sum_r \gamma_r^2 < \infty$ が成り立てば、学習の**収束が保証**される。
こういった方法は、**確率的近似**と呼ばれる手法である。

逆伝播のデメリットとして、収束の速度に**導関数が依存**しており、0に近づくと極めて遅くなる。

11.5.1. NNの訓練時の問題点：初期値

重み α, β が**ほぼ0**だとシグモイド関数が**狭い区間で線形**と見なせてしまう。

更に、重みが**完全に0**になると、微分値が0になり**重みが更新されなく**なってしまう。

重みの初期値を0付近のランダムな値を取ることは、以下の3つのメリットがある。

1. **重みベクトルが局在化**するように初期化できている
2. 必要に応じて**非線形性**が導入される
3. 望ましい値に収束しやすい

11.5.2. NNの訓練時の問題点：過学習

NNは**重みパラメータを多数**に含むため、誤差関数の大域的最小解は**過学習**になりがち
過学習の対策として、以下の2つの手法がある。

1. 早期停止
2. **荷重減衰**

線形モデルにおける**リッジ回帰**に相当手段である。

荷重減衰

誤差関数 $R(\theta)$ に罰則項 $J(\theta)$ を加え、**新たな誤差関数** $R(\theta) + \lambda J(\theta)$, $\lambda \geq 0$ を設計。
この時、 λ は**調整可能**なパラメータで、 λ を大きくすると重みは0になりやすくなる。
実用上では、 λ の値は**交差確認**によって推定する。
また、罰則項は以下の(11.16)式により与えられる。

$$J(\theta) = \sum_{k, m} \beta_{km}^2 + \sum_{m, l} \alpha_{ml}^2$$

これらの罰則項の他に、**荷重消去罰則項**も存在する。
上記の罰則項より、**重みを小さく**する効果が大い。

$$J(\theta) = \sum_{k, m} \frac{\beta_{km}^2}{1 + \beta_{km}^2} + \sum_{m, l} \frac{\alpha_{ml}^2}{1 + \alpha_{ml}^2} \quad (11.17)$$

11.5.3. NNの訓練時の問題点：入力のスケーリング

入力変数のスケーリングせずに利用すると、**最下層の重みへの影響**が大きくなり、結果的に最終的な出力にも影響を与える。

よって、入力変数は平均0、標準偏差1となるように**標準正規化**しておくべきである。

これによりランダムに決める**重みの初期値の範囲**を適切に設定することができる。

これは、経験則により $[-0.7, 0.7]$ と設定されることが多い。

11.5.4. NNの訓練時の問題点：隠れユニット/隠れ層の数

中間層のことを**隠れ層**といい、ここに含まれるユニットを**隠れユニット**という。
この隠れユニットの個数は、少なすぎるよりは**多すぎる方が望ましい**。

隠れユニットが少ない

- データの非線形性を吸収して柔軟性を持たせることが**難しい**

隠れユニットが多い

- 複雑な分類が可能になるが、過学習の要因になる
→ 正則化により、**余分な重みを0**にすれば過学習を防げる。

実用上では、最適なユニット数や正則化パラメータに対して、**交差確認**を利用する。

11.5.5. NNの訓練時の問題点：複数の極小解

誤差関数 $R(\theta)$ は非凸でかつ多数の極小解をもつ。

→ 重みの初期値の選択に依存

対策として、以下の3つの様な手法がある。

1. 重みの初期値の集合を用意して、最小の誤差を与えるものを最適解として選択
2. 重みの初期値を複数個用意して、得られた多数のネットワークの出力を平均化して、最終的な出力とする
3. バギングを利用して、訓練データの一部からNNを作成する工程を複数回行い、それらの予測値を平均化

11.7 畳み込みニューラルネットワーク(CNN)

CNNは畳み込み層とプーリング層、全結合層の3種の層を中間層内に持つNNである。

1. 畳み込み層

入力の一部をまとめて1つの入力とする層。

画像なら、 $N \times N$ ピクセル $\mathbf{P}$ とカーネル $\mathbf{K}$ の畳み込み $\mathbf{P} * \mathbf{K}$ を特徴量とする。

2. プーリング層

入力のサイズを削減する層。

後の層で扱いやすくするためにある。

画像内の各 $N \times N$ ピクセルごとに代表値を選択することでデータ量を削減する。

3. 全結合層

上記の2層の全ての要素と接続する層。

主に最後の判定を行う層で使われる。

11.8. PPRとNNの共通点

- 入力に対する線型結合(**導出特徴量**)が**非線形関数**の形
→ **信号対雑音比**が高いデータでも利用可
- 各入力が**モデルの様々な場所**で、非線形関数を通して取り込まれる。
→ データ生成の過程や解釈性に難あり

11.9. NNの性能評価

NNの性能評価では、以下のような手法を組み合わせで評価する。

- 推論性能(**ACC**) ... 正しく分類される確率
- **ROC曲線** ... **TPR**と**FPR**を元に作られた曲線
- **AUC** ... ROC曲線の右下の領域の面積
- **MAE/MSE** ... 真の値と予測値の差の**絶対値の平均**及び**2乗した値の平均**

11.9.1. ベイジアンニューラルネットワーク(BNN)

通常のNNでは、重みは学習の結果収束した定数である。

しかし、BNNでは重みは**ある確率分布に基づいて出力された確率変数**とする。

また、BNNに似ている手法に**ドロップアウト**というものがある。

これは、エポック毎に**中間層のユニットを一定確率で無効化する**手法である。

→ **ブースティング**に似ている

どちらも過学習防止の効果がある。

11.10. 計算上考慮すべき事柄

NNの当てはめには、 N 個の観測、 p 次元の入力変数、 M 個の隠れユニット、 L 個の訓練エポックより $O(NpML)$ の計算量が必要になる。

また、ソフトウェア毎の計算量や予測精度のばらつきが大きい点より、目的に応じたソフトウェア選択が必要である。

参考文献

[1] 射影追跡回帰 - Wikipedia (参照 : 2025/07/03)

<https://ja.wikipedia.org/wiki/射影追跡回帰>

[2] 初心者の初心者による初心者のためのニューラルネットワーク#1～理論：順伝播編～ #機械学習 - Qiita (参照 : 2025/07/10)

<https://qiita.com/sakigakeman/items/55f4cc1c50f8236ac7a6>

[3] 【初心者向け】ねこでも分かるニューラルネットワーク超入門 #Python - Qiita (参照 : 2025/07/10)

<https://qiita.com/NekoAllergy/items/489a4158b15231936f11>

[4] 何故ニューラルネットの予測能力は射影追跡回帰を上回るのか - jstage (参照 : 2025/07/10)

https://www.jstage.jst.go.jp/article/oukan/2007/0/2007_0_115/_pdf/-char/ja

[5] 誤差逆伝播法を完全に理解したので説明する (参照 : 2025/07/10)

https://zenn.dev/yuto_mo/articles/4607b7feb8ce8b

[6] 誤差逆伝搬法の理論を計算してみる (参照 : 2025/07/10)

https://zenn.dev/yuto_mo/articles/b56e1fc2215c8b

[7] Hidden Unit/Layer(隠れユニット/隠れ層)の基本 #機械学習 - Qiita (参照 : 2025/07/10)

<https://qiita.com/yuxzux/items/8c17a602047022637662>

[8] Convolutional Neural Networkとは何なのか #Python - Qiita (参照 : 2025/07/10)

<https://qiita.com/icoxfog417/items/5fd55fad152231d706c2>

[9] 畳み込みニューラルネットワーク(CNN)をわかりやすく基本から実装まで解説 – 体験型学習ブログ by zero to one (参照 : 2025/07/10)

https://zero2one.jp/learningblog/cnn-for-beginners/?srsId=AfmBOoofaQtM2NWskPMdzM9zItk1LfTHK1oQTB_aBxSMMQ9N37idTugp

[10] ニューラルネットワークにおけるエポック数の評価と最適化方法 (参照 : 2025/07/12)

<https://jp.linkedin.com/pulse/ニューラルネットワークにおけるエポック数の評価と最適化方法-太一-遠藤-pdfkc>

[11] AUCをちゃんと説明できないはずがない (参照 : 2025/07/12)

https://zenn.dev/d2c_mtech_blog/articles/38dd54be656ba9

[12] ベイジアンニューラルネットワークと予測の不確実性について #DeepLearning - Qiita (参照 : 2025/07/12)

https://qiita.com/qiita_kuru/items/8d20986b51c8e57e51b5#ベイジアンニューラルネットワーク